FRE-GY-9743 Assignment 5: Implicit Function Differentiation and a Valuation Engine

Raj Pawar, NYU Tandon School of Engineering, October 25, 2026.
Code: github.com/pawarraj8888/FRE-GY-9743-Assignments-5, branch `Raj_Pawar_Branch`.
Project page: pawarraj8888.github.io/FRE-GY-9743-Assignments-5.

Part I: implicit function differentiation

Setting. The model has an internal state $X_I \in \mathbb{R}^n$ (for a curve, the forward rates at the nodes; for a smile, the interpolator's parameters). The market gives me n quotes X_M, and for each quote the model has a pricing function $V_i(X_I)$. Calibrating means finding the X_I with $V(X_I) = X_M$: as many equations as unknowns.

Q1: solving $V(X_I) = X_M$. It is a square root-finding problem, $F(X_I) := V(X_I) - X_M = 0$, and my default tool is Newton's method. From a starting point $X_I^{(0)}$ (yesterday's calibration, or a flat state) I iterate

$$J\big(X_I^{(k)}\big)\,\Delta^{(k)} = -F\big(X_I^{(k)}\big), \qquad X_I^{(k+1)} = X_I^{(k)} + \lambda\,\Delta^{(k)}, \qquad J = \frac{\partial V}{\partial X_I},$$

solving the linear system rather than inverting J, with a line search on the step length $\lambda \le 1$, or Levenberg-Marquardt damping of the linear system, when I start far away, and bounds that keep the state admissible. I stop when $\|F\|$ is below a tolerance measured in quote units, a small fraction of a basis point or of a vol point. The Jacobian comes analytically when the model exposes $\partial V/\partial X_I$, which is what the gradient methods in this library are for; otherwise by finite differences, at n extra pricings per iteration. Two things make the problem easier in practice. If instrument i only depends on the first i state variables, J is lower triangular and Newton collapses into n one-dimensional root searches done in order: that is bootstrapping a curve node by node, with Brent or the secant method. In that bootstrapped case, if each V_i is monotone in its own node (a price in a vol, a swap rate in a forward rate), every one-dimensional solve has a single root and the curve is unique. In general I only get local uniqueness, and the derivative in the next question, when J is non-singular at the solution, which is the hypothesis of the implicit function theorem.

Q2: dX_I/dX_M at the solution. The calibrated state is a function of the quotes, $X_I^*(X_M)$, defined implicitly by $V\big(X_I^*(X_M)\big) = X_M$. Differentiating that identity with respect to X_M and using the chain rule,

$$\frac{\partial V}{\partial X_I}\,\frac{dX_I^*}{dX_M} = I \quad\Longrightarrow\quad \frac{dX_I^*}{dX_M} = J^{-1}, \qquad J = \frac{\partial V}{\partial X_I}\Big|_{X_I^*}.$$

That is the implicit function theorem: no re-calibration and no bumping of quotes, just the inverse of the Jacobian I already had from the last Newton step. Column j of J^{-1} says how every state variable moves when quote j moves by one unit with the others held fixed. It is also how model risk becomes market risk: for any trade with value $P(X_I)$,

$$\frac{dP}{dX_M} = \frac{\partial P}{\partial X_I}\,J^{-1},$$

so a vector of state sensitivities (what Part II computes) becomes risk in the quoted instruments after one linear solve, and a hedge ratio once each entry is divided by that instrument's own sensitivity to its quote (one for a price, the annuity for a par rate). In this assignment the state *is* the quote (the IFR nodes are the inputs), so $J = I$ and the risk report needs no Jacobian; a curve calibrated to swap rates would need the J^{-1} step. If J is nearly singular the entries of J^{-1} blow up, and the theorem is telling me that the calibration itself is unstable, not that the hedges are large.

Q3: three OTM options and a vol interpolator. The quotes are the undiscounted prices P_1, P_2, P_3 at $K_1 < K_{\text{atm}} < K_3$ (an OTM put, the ATM option, an OTM call), the state is $\theta \in \mathbb{R}^3$, and the model price of strike K_i is $V_i(\theta) = \text{Black}\big(F, K_i, T, \sigma(K_i;\theta)\big)$ without the discount factor. Three prices and three parameters, so the recipe from Q1 applies, with the chain rule for the Jacobian:

$$J_{ij} = \frac{\partial V_i}{\partial \theta_j} = \underbrace{\frac{\partial\,\text{Black}_i}{\partial \sigma}}_{\text{vega at } K_i}\,\frac{\partial \sigma(K_i;\theta)}{\partial \theta_j}.$$

I would not actually run Newton on prices, though. Black's price is strictly increasing in σ, so each P_i inverts to a unique implied vol σ_i^{mkt} with a one-dimensional solve, and the calibration becomes $\sigma(K_i;\theta) = \sigma_i^{\text{mkt}}$ for $i = 1, 2, 3$: the interpolator has to pass through three points. If the interpolator is linear in its parameters, say a quadratic in log-moneyness $x = \ln(K/F)$ with $\sigma(K;\theta) = \theta_0 + \theta_1 x + \theta_2 x^2$, this is a 3×3 linear system with the constant matrix

$[1,\ x_i,\ x_i^2]$, solved exactly in one step, and θ is unique whenever the three strikes are distinct. For a nonlinear family such as SABR with β fixed, $\theta = (\alpha, \rho, \nu)$, Newton takes a few iterations from a start read off the quotes (α from the ATM level, ρ from the slope, ν from the curvature), with $|\rho| < 1$ and $\nu > 0$ enforced.

How $\sigma(K)$ moves with θ is the gradient of the interpolator, $\partial\sigma(K;\theta)/\partial\theta$, one row of three numbers per strike; for the quadratic it is just the basis $(1, x, x^2)$. The more useful question is how the whole smile moves when a quote moves, and that is Q2 again: $d\sigma(K)/dP = \big(\partial\sigma(K)/\partial\theta\big)J^{-1}$, or in vol terms $d\sigma(K)/d\sigma^{\text{mkt}} = \big(\partial\sigma(K)/\partial\theta\big)\big[\partial\sigma(K_i;\theta)/\partial\theta\big]^{-1}$, three weights that say how much of each quoted option reproduces a vol move at K (for an interpolator with a pure level parameter, such as θ_0 here, they sum to one, because shifting the three quotes together shifts the whole smile).

What θ means to a trader. Nobody quotes $\theta_0, \theta_1, \theta_2$. The desk quotes the ATM vol, the risk reversal $\sigma(K_3)-\sigma(K_1)$ and the butterfly $\frac{1}{2}\big(\sigma(K_1)+\sigma(K_3)\big)-\sigma(K_{\text{atm}})$: level, slope and curvature of the smile. With three parameters and three quotes the map between them is one-to-one, and for symmetric strikes $x_3 = -x_1 = d$, $x_{\text{atm}} = 0$ it is explicit: ATM $= \theta_0$, RR $= 2\theta_1 d$, BF $= \theta_2 d^2$. So intuitively θ_0 is the level, θ_1 the skew (the tilt of the smile, the spot-vol correlation, ρ in SABR) and θ_2 the smile (its convexity, the vol of vol, ν in SABR). If I parameterize the interpolator directly in (ATM, RR, BF), then $\partial\sigma(K)/\partial\theta$ reads as "vol points at K per vol point of ATM, of RR and of BF", which is the form in which the trader hedges it with the three quoted options.

Part II: valuation engine and first-order risk

I completed the five `TODO`s. In `fixedincomelib/yield_curve/yield_curve_model.py`: (1) the component's local gradient. With $D_{\text{local}}(T) = \exp\big(-\int_0^T f\big)$ and f piecewise constant, $\partial D/\partial f_i = -D\,w_i$, where w_i is the length of node i's bucket inside $[0,T]$ and is exactly what `gradient_of_integrated_value_wrt_ordinate(0,T)` returns; the method writes $-\text{scaler}\cdot D\,w$ into the caller's array, replacing it or adding to it. (2) The model's gradient walks the reference chain the way `discount_factor` does. Since $D_{\text{funding}} = D_{\text{SOFR}}\,D_{\text{spread}}$, the product rule gives the SOFR block $D_{\text{spread}}\,\partial D_{\text{SOFR}}/\partial f$ and the funding block $D_{\text{SOFR}}\,\partial D_{\text{spread}}/\partial s$; I recurse with the running product carried in the scaler, and when overwriting I zero every block first so that a component off the chain comes back as zeros rather than stale numbers. In `fixedincomelib/yield_curve/valuation_engine.py`: (3) the constructor reads the `FUNDING INDEX PARAMETER`, resolves the funding identifier for the product's currency and stores $C = N\alpha$; (4) `calculate_value` sets $PV = C\,D(T_p)$ and cash 0 for a future payment, $PV = \text{cash} = C$ on the value date, and zeros for a past payment; (5) `calculate_risk` is C times the model gradient, with C passed through as the scaler, and zero blocks (or nothing added) when there is no discount factor to differentiate.

The notebook's case: value date 2026-01-02, payment 2026-10-02, so $T = 273/365 = 0.747945$. The SOFR IFR nodes sit at 0.246575, 1 and 1.999993 years (ACT/ACT ISDA, so the 2028 leap day pulls the last one just under 2) and the spread nodes at 0.495890 and 1 year, so the bucket lengths inside $[0,T]$ are (0.246575, 0.501370, 0) and (0.495890, 0.252055): the third SOFR node carries no risk, and the first spread node does even though its spread is zero. With $D_{\text{SOFR}} = 0.972921$, $D_{\text{spread}} = 0.999748$, $D_{\text{funding}} = 0.972676$ and $C = 1{,}000{,}000 \times 91/360 = 252,777.78$, the cashflow is worth $PV = 245,870.91$ with no cash today. The last column is the signed PV change for a one basis point rise in the node, the `UNIT` scaling the risk report applies on display.

Component	Node	IFR	w_i	$\partial D_{\text{local}}/\partial f_i$	$\partial D_{\text{funding}}/\partial f_i$	$\partial PV/\partial f_i$	per bp
SOFR-1B	2026-04-02	0.030	0.246575	-0.239898	-0.239838	-60,625.70	-6.0626
SOFR-1B	2027-01-02	0.040	0.501370	-0.487793	-0.487670	-123,272.26	-12.3272
SOFR-1B	2028-01-02	0.050	0.000000	0.000000	0.000000	0.00	0.0000
SOFR-1B-FLAT	2026-07-02	0.000	0.495890	-0.495765	-0.482341	-121,925.03	-12.1925
SOFR-1B-FLAT	2027-01-02	0.001	0.252055	-0.251991	-0.245168	-61,972.94	-6.1973

The notebook runs top to bottom with the professor's cells untouched; its printed gradients, PV and risk table agree with the hand calculation above at the precision shown. `tests/test_yield_curve_gradient.py` (16 tests) checks the local and model gradients against the bucket-length formula and against central bump-and-reval through `perturb_model_parameter`, the product rule, the overwrite and accumulate semantics, block allocation, serialization and the validation errors; `tests/test_valuation_engine.py` (12 tests) checks the engine's PV, cash and risk for future, same-day and past payments, the sign of a short position, the registry dispatch and the PV and risk reports.